

Step 1 - Main Idea of Initiative

This project proposes a Minimum-Nights Optimization model for Airbnb that recommends suitable minimum-nights settings for each Airbnb listing in Montreal. The objective of the model is to determine the minimum-night requirement that can improve expected revenue while upholding strong occupancy levels. Using two main sources of information: listings.csv, containing details about the room type, amenities, location, and minimum-night settings, and calendar.csv which provides day-level availability information. Using these inputs from these datasets, the model produces different minimum night scenarios and returns the highest expected revenue for each listing, computing the proportion of booked days within a 30-day window.

To account for demand fluctuations such as seasonality and temporary demand surges (holiday, special events, etc), we incorporate these patterns using the information found directly in the calendar.csv dataset, in order to ensure that our model remains fully aligned with our dataset. The model will then perform simulations over candidate values, with the objective of modeling the minimum_nights setting based on how it impacts expected nights booked. The model will then relay a recommendation such as: “Reduce minimum nights from 5 to 3 during off-peak periods to increase occupancy and expected revenue”. The model optimizes Airbnb efficiency by optimizing an important decision variable that is currently being set manually by the host.

Step 2 - Benefits provided by our model

If implemented, the new change could result in higher revenue and gross merchandise value through increased bookings, or “stays,” on a listing. Lower minimum-night requirements can increase booking opportunities by making listings more accessible to a wider range of guests, particularly shorter term and weekend visitors. As a result, the model improves occupancy rates, increases booking conversion, while generating higher transaction volume and revenue on the

platform. It also improves the user experience by reducing ‘available but not bookable’ cases and ensuring that the booking process is more seamless, thus increasing user satisfaction. In our supply side version, this model can be used to give data-driven recommendations to hosts about what they can do to improve their occupancy and pricing stability. This is a very effective way to maintain engagement with the platform and assure price consistency with similar bookings.

Finally, the model would improve matching efficiency across different types of demand, such as weekend trips or short-term visits. The outputs given by the model are made to adapt and be flexible towards various demand patterns that are taken directly from our datasets.

Step 3 - Implementation strategy

Since listings.csv contains listing-level information while calendar.csv records daily availability, the two files first need to be cleaned and combined into one consistent structure. In listings.csv, we clean the price variable by removing symbols and converting it into numeric form. We also cap minimum_nights at 30 to reduce the influence of very extreme values, while still keeping the raw value for reference. Missing values in variables such as beds, bathrooms, price, and review_scores_rating are filled using neighbourhood medians and then overall medians when necessary. In addition, we create useful features such as superhost_binary, review_imputed_flag, log-transformed price and accommodates, and several amenity indicators like WiFi, kitchen, washer, air conditioning, free parking, pool, and gym.

Once the listing data are cleaned, we prepare calendar.csv to capture short-term booking activity. We convert the date field, remove duplicates, and define a booked variable by treating available = 'f' as unavailable or booked. Each record is then assigned to Peak, Shoulder, or Off-Peak season. Using this cleaned calendar data, we calculate booking_rate_30 for each listing-season pair as booked days divided by total days in the 30-day window. We also create a demand_spike_flag by

comparing each listing's booking rate with the neighbourhood-season average. Finally, we merge these season-level booking measures with the cleaned listing features to form the final modelling dataset.

With the data prepared, we use two regression techniques to study how `minimum_nights` affects booking performance: Linear Regression and Random Forest Regression. Linear Regression is included as a baseline because it is simple and easy to interpret. It allows us to observe the general relationship between `minimum_nights` and `booking_rate_30` while controlling for other listing characteristics. However, this model assumes that the effect of `minimum_nights` is linear, which may not be realistic in this context. For example, increasing the minimum stay from 1 to 2 nights is probably not equivalent to increasing it from 7 to 14 nights.

Because of this limitation, our main model is Random Forest. Compared with Linear Regression, Random Forest is more suitable for this project because it can capture non-linear relationships and interactions across variables such as price, season, review score, and minimum-night rules. This is important because booking outcomes are likely influenced by multiple factors at the same time rather than by one simple linear effect. We implement the analysis in Python using pandas and numpy for data cleaning and feature engineering, matplotlib for visualization, and scikit-learn for model training, cross-validation, and hyperparameter tuning.

For Linear Regression, there is not much hyperparameter tuning involved, so the main focus is on selecting the right predictors and creating dummy variables for categorical variables such as room type, season, and neighbourhood. For Random Forest, parameter choice matters much more, so we use grid search with 5-fold cross-validation. The parameter grid includes `n_estimators = [100, 200]`, `max_depth = [10, None]`, `min_samples_leaf = [5, 10]`, and `max_features = ["sqrt", 0.5]`. These values are chosen because they help control the balance between flexibility and overfitting.

Based on the cross-validation results, the best Random Forest model uses 200 trees, no depth limit, a minimum leaf size of 5, and `max_features = 0.5`.

Finally, after fitting both models, we select the final model based on predictive performance. We compare them using 5-fold cross-validated R^2 and RMSE. Linear Regression gives a CV R^2 of 0.1672 and a CV RMSE of 0.4005, while Random Forest performs much better with a CV R^2 of 0.3610 and a CV RMSE of 0.3508. Since Random Forest has both higher explanatory power and lower prediction error, we choose it as the final model. This choice is also supported by the feature importance results, where `minimum_nights` remains one of the more important predictors alongside variables such as `log_price`, `review_scores_rating`, `season_Peak`, and `demand_spike_flag`.

After selecting Random Forest as the final model, we use it for recommendation simulation. For each listing-season observation, all variables are kept fixed except `minimum_nights`, which is replaced by candidate values such as 1, 2, 3, 4, 5, 7, 14, and 30. For each candidate, we predict a new booking rate and convert it into estimated 30-day revenue. The recommended minimum-night rule is the one that produces the highest expected revenue. In this way, the project does not stop at prediction alone, but directly turns the model into a practical decision-support tool for Airbnb listings.

Step 4 - Post-Implementation Strategy

4.1 Enhancing Performance with More Data

The current model relies solely on the existing dataset, but its predictive accuracy could be significantly improved by incorporating additional data sources.

The key addition will be introducing the real booking transaction record to replace the rate extracted from the `unavailable_date`. The implementation would correct the current inflated

estimates caused by failing to account for host-blocked dates. External data sources can further increase the model's reliability. For example, the event data from Ticketmaster API or Tourism Montréal help forecast demand surges, while the weather data from Environment Canada clarifies whether the low occupancy is caused by restrictive settings or by unfavourable conditions. Lastly, host-reported cost data would shift the model from revenue optimization to profit optimization and ensure recommendations account for maintenance costs.

4.2 Validation with Market Test

The model should be validated through a controlled A/B test before the official launch. A randomly selected group of hosts would be provided with the `minimum_night` recommendation, while the other group would continue to manage their settings manually. To confirm value, we define the following hypotheses on booking rate and revenue using a significance threshold of $\alpha = 0.05$:

$$H_0: \mu_{\text{treatment}} = \mu_{\text{control}} \text{ (no effect)}$$

$$H_1: \mu_{\text{treatment}} > \mu_{\text{control}} \text{ (higher performance)}$$

The result would then determine whether the model is practical or needs to be refined.

4.3 Quarterly Upgraded

As Airbnb's listing landscape changes continuously, next year's demand patterns can differ significantly from this year's. Therefore, the model should be seen as a dynamic system and evolving alongside the market. Hence, a quarterly upgrade with the most recent data is necessary for more accurate and current recommendations.